Serifa

Banco de dados

Por que PostgreSQL no servidor e IndexedDB no navegador, diagrama de relacionamentos, tabelas, object stores, revisões, concorrência otimista, consultas prontas, backup e restauração, integridade e segurança e preparação do PostgreSQL.

Copyright (©) 2026 Clara Portes (Violet-pixel)
Versão online: https://violet-pixel.github.io/Fiction-Writing-Editor/

Sumário

Visão geral **4**

Por que PostgreSQL no modo servidor 4

Por que IndexedDB no navegador 4

Áreas do modelo 5

Modo servidor sem PostgreSQL 5

Diagrama de relacionamentos **6**

Regras e convenções **7**

Convenções do modelo 7

Integridade 7

Exclusões em cascata 8

Validação antes de gravar 8

Tabelas **9**

projeto 9

capitulo 10

revisao 10

dicionario_pessoal 11

ideia_ignorada 11

sessao_escrita 11

exportacao 12

preferencia 12

Navegador: IndexedDB **13**

Como cada store espelha uma tabela 13

Abertura e atualização do banco 14

Versões e revisões **15**

Cada salvamento guarda uma revisão 15

Restaurar uma revisão 15

Concorrência otimista **16**

O que o editor faz num conflito 16

Consultas prontas **17**

Palavras por dia 17

Capítulos por status 17

Últimas revisões 17

Exportações por formato 17

Projetos recentes com total de palavras 18

Espaço ocupado 18

Backup e restauração **19**

Modo servidor: pg_dump 19

Qualquer modo: exportar .serifa 19

Integridade e segurança **20**

O que o banco guarda (e o que não guarda) 20

Proteções no servidor 20

Usuário do banco com o mínimo de permissões 20

Preparar o PostgreSQL **21**

Variáveis de ambiente 21

Solução de problemas 22

Capítulo 1

Visão geral

O Serifa guarda **somente o que a pessoa escreve** (projetos, capítulos, notas e revisões) e as **preferências** do editor. As mesmas entidades existem em dois lugares, conforme a maneira de abrir o programa:

Como o Serifa foi aberto	Onde os dados ficam	Quem grava
Site no GitHub Pages	IndexedDB do navegador (banco `serifa`)	O próprio editor, em JavaScript
`visualizar.bat` (servidor local pelo PowerShell)	IndexedDB do navegador (banco `serifa`)	O próprio editor, em JavaScript
Servidor Java com a API	PostgreSQL, ou arquivos JSON em `SERIFA_DATA` quando não há banco configurado	A API (`/api/v1/projects`)

Por que PostgreSQL no modo servidor

- **Transações de verdade.** Salvar um projeto troca o projeto, os capítulos e grava uma revisão numa única transação: ou tudo entra, ou nada entra. Nunca fica um capítulo pela metade.
- **Regras no próprio banco.** Chaves estrangeiras, `CHECK` e índices únicos barram dados inválidos mesmo que um cliente com defeito tente gravá-los.
- **Texto longo e JSON nativos.** `text` guarda capítulos inteiros sem limite prático e `jsonb` guarda o documento `.serifa` de cada revisão já validado e consultável.
- **Backup simples.** `pg_dump` gera uma cópia consistente com o servidor ligado (capítulo 9).
- **Sem dependência de compilação.** O servidor carrega o driver JDBC por reflexão; sem `SERIFA_DB_URL`, ele usa arquivos JSON e funciona do mesmo jeito.

Por que IndexedDB no navegador

No GitHub Pages e no `visualizar.bat` não há servidor de aplicação: só arquivos estáticos. O IndexedDB é o banco que todo navegador moderno já traz, por isso o Serifa funciona sem instalar nada:

- guarda centenas de megabytes (bem mais que o `localStorage`), suficiente para muitos livros e revisões;
- é transacional e assíncrono: o salvamento automático não trava a digitação;
- tem índices (`projetoId`) para listar os capítulos e as revisões de um projeto sem ler tudo;
- os dados ficam **no computador**, presos à origem do site; nada é enviado para a internet.

> **Importante.** Cada origem tem o seu próprio IndexedDB. O que foi escrito no site do GitHub Pages não aparece no `visualizar.bat` (que abre em `http://localhost`) e vice-versa. Para levar um projeto de um para o outro, use **Arquivo › Salvar como** (`.serifa`) e **Arquivo › Abrir**.

Áreas do modelo

Área	Tabelas	Guarda
Escrita	`projeto`, `capitulo`	O livro ou fanfic, a configuração de página e cada capítulo com HTML, notas e status
Histórico	`revisao`	Cópia completa do projeto a cada salvamento (até 50 por projeto)
Revisão de texto	`dicionario_pessoal`, `ideia_ignorada`	Palavras aceitas pelo corretor e balões de ideia que a pessoa mandou ignorar
Estatísticas	`sessao_escrita`, `exportacao`	Palavras escritas por sessão (meta diária) e registro das exportações
Interface	`preferencia`	Tema, corretor e balões ligados ou desligados, modo foco e outras opções

Modo servidor sem PostgreSQL

Quando `SERIFA_DB_URL` não está definida, o servidor Java grava cada projeto como um arquivo JSON (o mesmo formato `.serifa`) na pasta `SERIFA_DATA` (padrão `./dados-servidor`), com gravação atômica: escreve num arquivo temporário e só depois troca pelo definitivo. As revisões ficam numa subpasta por projeto, com o mesmo limite de 50. As regras deste documento valem para os dois modos; o PostgreSQL acrescenta as garantias do próprio banco.

Capítulo 2

Diagrama de relacionamentos

Tudo gira em torno de `projeto`. Apagar um projeto apaga, em cascata, os capítulos, as revisões, as palavras do dicionário daquele projeto, as ideias ignoradas, as sessões e o registro de exportações. `preferencia` é independente: vale para o editor inteiro.

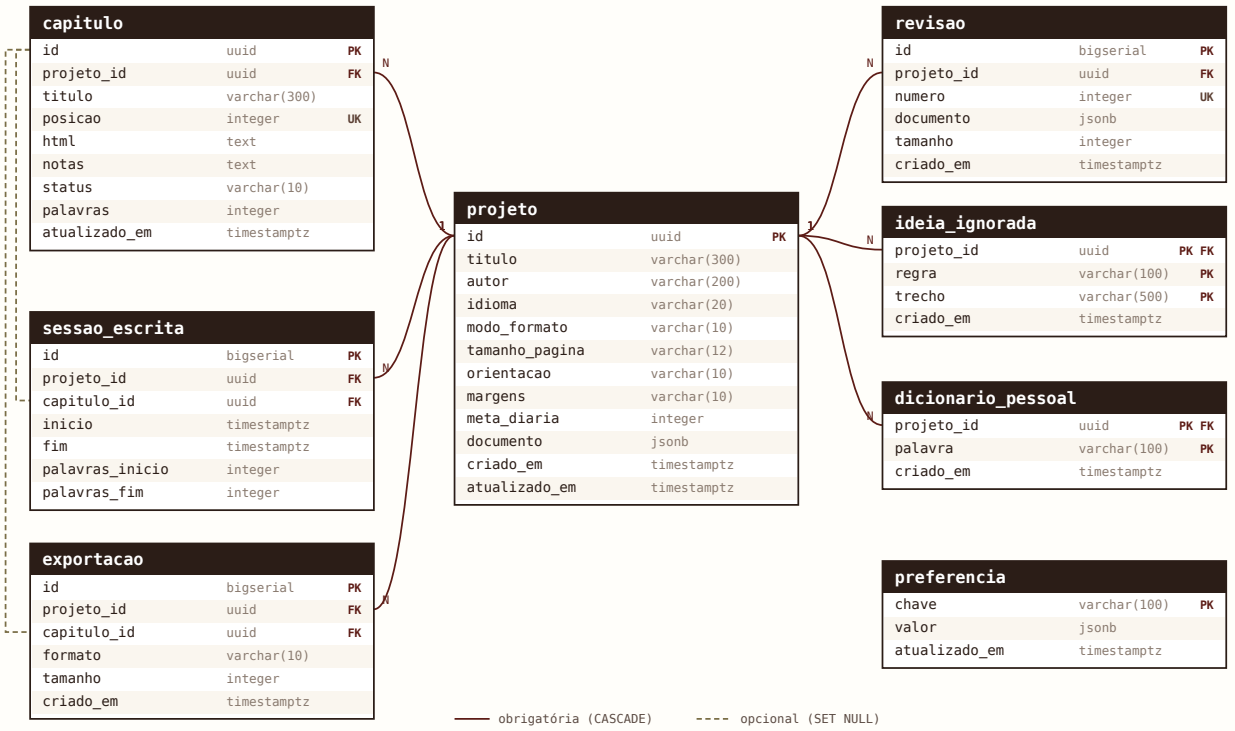


Figura 1 · projeto 1:N capitulo · projeto 1:N revisao · projeto 1:N sessao_escrita · projeto 1:N exportacao · projeto 1:N ideia_ignorada · projeto 1:N dicionario_pessoal · capitulo 0..1:N sessao_escrita / exportacao (SET NULL)

Relação	Cardinalidade	Ao apagar o pai
`capitulo.projeto_id` → `projeto.id`	1 projeto : N capítulos	`ON DELETE CASCADE`
`revisao.projeto_id` → `projeto.id`	1 : N (máx. 50 guardadas)	`ON DELETE CASCADE`
`dicionario_pessoal.projeto_id` → `projeto.id`	1 : N	`ON DELETE CASCADE`
`ideia_ignorada.projeto_id` → `projeto.id`	1 : N	`ON DELETE CASCADE`
`sessao_escrita.projeto_id` → `projeto.id`	1 : N	`ON DELETE CASCADE`
`sessao_escrita.capitulo_id` → `capitulo.id`	0..1 : N	`ON DELETE SET NULL`
`exportacao.projeto_id` → `projeto.id`	1 : N	`ON DELETE CASCADE`
`exportacao.capitulo_id` → `capitulo.id`	0..1 : N	`ON DELETE SET NULL`

Capítulo 3

Regras e convenções

Convenções do modelo

- **Nomes em português, `snake_case`** e no singular: `capitulo`, `sessao_escrita`, `atualizado_em`.
- **`uuid` para o que nasce no navegador** (`projeto`, `capitulo`): o editor cria o identificador sem falar com o servidor, então um projeto feito no GitHub Pages pode ser enviado para a API mais tarde sem trocar de id.
- **`bigserial`** para registros auxiliares (revisões, sessões, exportações, dicionário).
- **Datas sempre em `timestamptz`** (UTC no banco; o editor mostra no fuso local). Para agrupar por dia, use `inicio::date` (ou `inicio AT TIME ZONE 'America/Sao_Paulo'`).
- **Valores fixos com `CHECK`** em vez de tipos `ENUM`: adicionar um formato novo é só trocar a restrição, sem migração de tipo.
- **Os mesmos valores do arquivo `.serifa`**: `rascunho`, `revisao`, `pronto`; `livro`, `wattpad`, `ao3`; `a4`, `livro-6x9` etc.

Integridade

Regra	Como o banco garante
Todo capítulo pertence a um projeto	`capitulo.projeto_id NOT NULL` + chave estrangeira
Dois capítulos não ocupam a mesma posição	`UNIQUE (projeto_id, posicao) DEFERRABLE INITIALLY DEFERRED`: permite reordenar todos numa transação
Título nunca vazio	`CHECK (length(btrim(titulo)) > 0)`
Status, formato, folha e margens conhecidos	`CHECK (... IN (...))` em `modo_formato`, `tamanho_pagina`, `orientacao`, `margens`, `status` e `formato`
Contadores nunca negativos	`CHECK (palavras >= 0)`, `CHECK (meta_diaria >= 0)`, `CHECK (tamanho >= 0)`
Revisão única por número	`UNIQUE (projeto_id, numero)`
Projeto sempre no formato certo	`CHECK (documento ->> 'format' = 'serifa')` em `projeto`
Palavra do dicionário sem repetição	`PRIMARY KEY (projeto_id, palavra)` + índice único `(projeto_id, lower(palavra))`
Ideia ignorada uma vez só	`PRIMARY KEY (projeto_id, regra, trecho)`
Datas coerentes	`CHECK (atualizado_em >= criado_em)` em `projeto`
Sessão termina depois de começar	`CHECK (fim IS NULL OR fim >= inicio)`

Exclusões em cascata

Apagar um projeto (`DELETE /api/v1/projects/{id}` ou **Excluir projeto** na lista de projetos) remove tudo o que é dele. Apagar um capítulo mantém as sessões e exportações, com `capitulo_id` nulo (`ON DELETE SET NULL`).

> **Vínculos preservados.** A cada salvamento, o servidor atualiza os capítulos no lugar (`INSERT ... ON CONFLICT (id) DO UPDATE`) e apaga só os que saíram do projeto, então sessões e exportações continuam ligadas aos capítulos que ainda existem.

Validação antes de gravar

1. A API confere o corpo: JSON válido, até 10 MB, `format = "serifa"` e `version = 1`.
2. O HTML de cada capítulo passa pelo sanitizador (só os blocos e marcações da especificação; nada de `script`, eventos `on*` ou links `javascript:`).
3. A contagem de palavras usa o mesmo contador do editor, que dá o mesmo resultado em JavaScript, Java e C.
4. Só então abre a transação, grava e cria a revisão (capítulo 6).

Capítulo 4

Tabelas

Conforme `server/src/main/resources/db/schema.sql` (PostgreSQL). Na coluna Nulo, "sim" indica que o campo aceita `NULL`. Todas as tabelas são criadas com `CREATE TABLE IF NOT EXISTS`, então o arquivo pode ser aplicado de novo sem erro.

projeto

Um livro, conto ou fanfic. `documento` guarda o projeto `.serifa` inteiro e é a fonte da verdade; as outras colunas são cópias para consulta e para os checks.

Campo	Tipo	Nulo	Descrição
`id` `PK`	`uuid`	não	Chave primária, gerada pelo editor
`titulo`	`varchar(300)`	não	Título; `CHECK (length(btrim(titulo)) > 0)`
`autor`	`varchar(200)`	não	Pseudônimo opcional; padrão `''` (vazio = sem autor)
`idioma`	`varchar(20)`	não	Padrão `pt-BR`
`modo_formato`	`varchar(10)`	não	`livro` (padrão), `wattpad` ou `ao3`
`tamanho_pagina`	`varchar(12)`	não	`a4`, `a5` (padrão), `carta`, `oficio`, `livro-6x9`, `bolso-5x8`, `b5`
`orientacao`	`varchar(10)`	não	`retrato` (padrão) ou `paisagem`
`margens`	`varchar(10)`	não	`normal` (padrão), `estreita`, `moderada`, `larga`, `espelhada`
`meta_diaria`	`integer`	não	Palavras por dia; padrão 1000; `CHECK (meta_diaria >= 0)`
`documento`	`jsonb`	não	Projeto completo; `CHECK (documento ->> 'format' = 'serifa')`
`versao`	`integer`	não	Concorrência otimista; padrão 1, sobe a cada gravação; `CHECK (versao >= 1)`
`criado_em`	`timestamptz`	não	Criação (padrão `now()`)
`atualizado_em`	`timestamptz`	não	Último salvamento (padrão `now()`)

Índices: `projeto_pkey (id)`, `projeto_atualizado_idx (atualizado_em DESC)` para a lista "Projetos recentes". **Checks:** os de cada coluna e `atualizado_em >= criado_em`. Folha, fonte, entrelinha, recuo e símbolo de cena ficam só em `documento -> 'settings'`.

> **Serifa para Windows.** O `Serifa.exe` usa o mesmo IndexedDB (na janela própria, em `%AppData%\Serifa\janela`) e grava, além disso, arquivos comuns: o `.serifa` de cada história e as exportações ficam em `<pasta base>\<folder>\`, e a pasta base fica em `%AppData%\Serifa\config.json` (`base`, `criarPasta`, `salvarAuto`). Nenhuma tabela nova é necessária no servidor: `folder` viaja dentro de `documento`.

Descrição da história. A descrição opcional (sinopse, fandom, avisos) fica em `documento ->> 'description'` (até 2.000 caracteres, vazia quando não há). Para listar as histórias com a descrição: `SELECT titulo, documento ->> 'description' AS descricao FROM projeto ORDER BY atualizado_em DESC;`

Coluna versao. `versao integer NOT NULL DEFAULT 1 CHECK (versao >= 1)` sobe 1 a cada gravação e é a base da concorrência otimista (capítulo 7).

capitulo

Cada capítulo do projeto, na ordem do painel de capítulos. O texto fica como HTML sanitizado.

Campo	Tipo	Nulo	Descrição
`id` PK	`uuid`	não	Chave primária, gerada pelo editor
`projeto_id` FK	`uuid`	não	Projeto dono (`ON DELETE CASCADE`)
`titulo`	`varchar(300)`	não	Nome do capítulo (padrão vazio; não entra no corpo exportado)
`posicao` UK	`integer`	não	Ordem, começando em 0; `CHECK (posicao >= 0)`
`html`	`text`	não	Conteúdo sanitizado (padrão vazio); um parágrafo pode levar a classe de estilo `estilo-pensamento`, `estilo-flashback`, `estilo-mensagem`, `estilo-carta` ou `estilo-nota` (outras classes são descartadas)
`notas`	`text`	não	Anotações do capítulo, nunca exportadas
`status`	`varchar(10)`	não	`rascunho` (padrão), `revisao` ou `pronto`
`palavras`	`integer`	não	Contagem de palavras; padrão 0; `CHECK (palavras >= 0)`
`atualizado_em`	`timestamptz`	não	Última alteração (padrão `now()`)

Chaves e índices: `UNIQUE (projeto_id, posicao) DEFERRABLE INITIALLY DEFERRED` (reordenar todos numa transação), `capitulo_projeto_idx (projeto_id, posicao)`.

revisao

Fotografia completa do projeto a cada salvamento, para voltar atrás. O servidor mantém as 50 últimas.

Campo	Tipo	Nulo	Descrição
`id` PK	`bigserial`	não	Identidade gerada pelo banco
`projeto_id` FK UK	`uuid`	não	Projeto (`ON DELETE CASCADE`)
`numero` UK	`integer`	não	Sequência no projeto (maior + 1); `CHECK (numero > 0)`
`documento`	`jsonb`	não	Projeto inteiro no formato `.serifa`
`tamanho`	`integer`	não	Tamanho do documento; `CHECK (tamanho >= 0)`

Campo	Tipo	Nulo	Descrição
`criado_em`	`timestamptz`	não	Quando a revisão foi feita (padrão `now()`)

Chaves e índices: `UNIQUE (projeto_id, numero)`, `revisao_projeto_idx (projeto_id, numero DESC)`.

dicionario_pessoal

Palavras que a pessoa adicionou ao dicionário do projeto (nomes de personagens, lugares, termos inventados). A comparação não diferencia maiúsculas.

Campo	Tipo	Nulo	Descrição
`projeto_id` PK FK	`uuid`	não	Projeto (`ON DELETE CASCADE`)
`palavra` PK	`varchar(100)`	não	A palavra; `CHECK (length(btrim(palavra)) > 0)`
`criado_em`	`timestamptz`	não	Quando foi adicionada (padrão `now()`)

Chaves e índices: `PRIMARY KEY (projeto_id, palavra)` e índice único `dicionario_pessoal_lower_idx (projeto_id, lower(palavra))`, que impede "Elara" e "elara" no mesmo projeto.

ideia_ignorada

Balões de ideia que a pessoa mandou ignorar. No `.serifa` é a lista `ignored` com textos `regra:trecho`; aqui as duas partes ficam em colunas separadas.

Campo	Tipo	Nulo	Descrição
`projeto_id` PK FK	`uuid`	não	Projeto (`ON DELETE CASCADE`)
`regra` PK	`varchar(100)`	não	Id da regra, ex.: `pleonasmo-subir-para-cima`
`trecho` PK	`varchar(500)`	não	Trecho do texto, ex.: `subiu para cima`
`criado_em`	`timestamptz`	não	Quando foi ignorada (padrão `now()`)

Chave: `PRIMARY KEY (projeto_id, regra, trecho)`.

sessao_escrita

Cada período de escrita. Alimenta "palavras da sessão", a meta diária e as estatísticas por dia (`inicio::date`).

Campo	Tipo	Nulo	Descrição
`id` PK	`bigserial`	não	Identidade
`projeto_id` FK	`uuid`	não	Projeto (`ON DELETE CASCADE`)
`capitulo_id` FK	`uuid`	sim	Capítulo da sessão (`ON DELETE SET NULL`)
`inicio`	`timestamptz`	não	Começo da sessão

Campo	Tipo	Nulo	Descrição
`fim`	`timestamptz`	sim	Fim (nulo enquanto a sessão está aberta)
`palavras_inicio`	`integer`	não	Total de palavras ao começar; padrão 0; `>= 0`
`palavras_fim`	`integer`	não	Total ao terminar; padrão 0; `>= 0`

Índices: `sessao_escrita_projeto_idx (projeto_id, inicio DESC)`. **Checks:** `fim IS NULL OR fim >= inicio`.

exportacao

Registro de cada exportação feita pela API. Não guarda o arquivo gerado: só o que foi exportado e quando.

Campo	Tipo	Nulo	Descrição
`id` PK	`bigserial`	não	Identidade
`projeto_id` FK	`uuid`	não	Projeto (`ON DELETE CASCADE`)
`capitulo_id` FK	`uuid`	sim	Capítulo exportado (`ON DELETE SET NULL`); nulo = projeto inteiro
`formato`	`varchar(10)`	não	`wattpad`, `ao3`, `txt`, `md`, `html`, `docx`, `epub`
`tamanho`	`integer`	não	Tamanho do arquivo gerado; `CHECK (tamanho >= 0)`
`criado_em`	`timestamptz`	não	Quando foi exportado (padrão `now()`)

Índices: `exportacao_projeto_idx (projeto_id, criado_em DESC)`.

preferencia

Preferências da instalação (tema, idioma da interface, atalhos), no formato chave e valor.

Campo	Tipo	Nulo	Descrição
`chave` PK	`varchar(100)`	não	Nome da opção; `CHECK (chave ~ '^[a-z0-9._-]+$')` (só minúsculas)
`valor`	`jsonb`	não	Valor (texto, número, booleano ou objeto)
`atualizado_em`	`timestamptz`	não	Última mudança (padrão `now()`)

Chave	Exemplo de valor	Para que serve
`tema`	`"claro"` / `"escuro"`	Tema da interface (`Ctrl`+`Shift`+`D`)
`corretor`	`true`	Corretor ortográfico ligado (`F7`)
`ideias`	`true`	Balões de ideia ligados (`Shift`+`F7`)
`ideias.categorias`	`["crase","pleonasmo"]`	Categorias de ideia ativas
`ultimo-projeto`	`"uuid"`	Projeto reaberto ao iniciar

Capítulo 5

Navegador: IndexedDB

No GitHub Pages e no `visualizar.bat` o editor abre o banco `serifa` (versão 1) do IndexedDB. Os *object stores* têm os mesmos campos das tabelas, com nomes em `camelCase`, iguais aos do arquivo `.serifa`.

Object store	Chave	Índices	Espelha a tabela
`projetos`	`id` (uuid)	—	`projeto` (+ `ideia_ignorada` na lista `ignored`)
`capitulos`	`id` (uuid)	`projetoId`	`capitulo`
`revisoes`	`id` (autoincremento)	`projetoId`	`revisao`
`dicionario`	`palavra` (minúscula)	—	`dicionario_pessoal` (geral)
`sessoes`	`dia` (`AAAA-MM-DD`)	—	`sessao_escrita` (um registro por dia)
`preferencias`	`chave`	—	`preferencia`

Como cada store espelha uma tabela

Store · campo	Tabela · coluna
`projetos` · `id, title, description, folder, author, language, createdAt, updatedAt, versao, words, chapterCount`	`projeto` · `id, titulo, autor, idioma, criado_em, atualizado_em` (e `versao`, capítulo 7); `description` fica em `documento ->> 'description'`; `words` e `chapterCount` são resumos para a tela Minhas histórias, recalculados a cada gravação
`projetos` · `folder`	`projeto.documento ->> 'folder'`: nome da pasta da história no Serifa para Windows (definido ao criar a história, até 120 caracteres, sem `\ / : * ? " < > |`)
`projetos` · `settings` (`formatMode, pageSize, orientation, margins, paper, fontFamily, fontSize, lineHeight, firstLineIndent, dailyGoal, sceneBreak`)	`projeto.documento -> 'settings'`; cópias em `modo_formato`, `tamanho_pagina`, `orientacao`, `margens`, `meta_diaria`
`projetos` · `ignored[]`, `dictionary[]`	`ideia_ignorada.regra` + `trecho` (o texto `regra:trecho` dividido), `dicionario_pessoal.palavra`
`capitulos` · `id, projetoId, title, position, html, notes, status, words, updatedAt`	`capitulo` · `id, projeto_id, titulo, posicao, html, notas, status, palavras, atualizado_em`
`capitulos` · `summary, pov, goal, createdAt` (painel do capítulo)	`projeto.documento -> 'chapters' -> n`: resumo (até 2.000 caracteres), ponto de vista (até 120), meta de palavras do capítulo (0 = sem meta) e data de criação
`revisoes` · `id, projetoId, documento, palavras, motivo, criadoEm`	`revisao` · `id, projeto_id, documento, criado_em`; `palavras` e `motivo` só existem no navegador (lá o documento é o JSON em texto)
`dicionario` · `palavra, criadoEm`	Dicionário geral, só no navegador (no servidor, o dicionário é por projeto)

Store · campo	Tabela · coluna
`sessoes` · `dia, projetoId, palavrasInicio, palavrasFim, acumulado`	`sessao_escrita` · `inicio::date, projeto_id, palavras_inicio, palavras_fim`; `acumulado` (só no navegador) soma o que foi escrito em outro projeto no mesmo dia
`preferencias` · `chave, valor` (chaves `prefs` e `lastProject`)	`preferencia` · `chave, valor` (no servidor, chaves só em minúsculas)

> **Exportações no navegador.** Sem servidor, as exportações não são registradas: o arquivo vai direto para a pasta escolhida ou para Downloads. A tabela `exportacao` só existe no modo servidor.

Abertura e atualização do banco

```
const req = indexedDB.open('serifa', 1);
req.onupgradeneeded = () => {
  const db = req.result;
  db.createObjectStore('projetos', { keyPath: 'id' });
  db.createObjectStore('capitulos', { keyPath: 'id' }).createIndex('projetoId', 'projetoId');
  db.createObjectStore('revisoes', { keyPath: 'id', autoIncrement: true }).createIndex('projetoId',
'projetoId');
  db.createObjectStore('dicionario', { keyPath: 'palavra' });
  db.createObjectStore('sessoes', { keyPath: 'dia' });
  db.createObjectStore('preferencias', { keyPath: 'chave' });
};
```

- Salvar um projeto usa **uma transação** `readwrite` sobre `projetos`, `capitulos` e `revisoes`: os três mudam juntos ou nenhum muda.
- Uma versão nova do banco (2, 3…) só acrescenta stores e índices dentro de `onupgradeneeded`; dados existentes nunca são apagados na atualização.
- Se o navegador bloquear o IndexedDB (janela anônima de alguns navegadores), o editor passa a trabalhar só na memória: nesse caso, salve o `.serifa` antes de fechar.

Capítulo 6

Versões e revisões

Cada salvamento guarda uma revisão

Ao salvar (`Ctrl`+`S`, salvamento automático ou `PUT /api/v1/projects/{id}`), o Serifa grava o projeto e, na mesma transação, uma linha em `revisao` com o documento `.serifa` completo. São mantidas as **50 revisões mais recentes** de cada projeto; as mais antigas são apagadas no mesmo salvamento.

1. Confere a `versao` esperada (capítulo 7) e incrementa `projeto.versao`.
2. Grava `projeto` (`INSERT ... ON CONFLICT (id) DO UPDATE`), atualiza os capítulos no lugar (`ON CONFLICT (id) DO UPDATE`) e apaga só os que saíram.
3. Insere a revisão com o próximo `numero` do projeto (maior número + 1).
4. Apaga o que passou de 50 e faz o `COMMIT`.

```
-- poda: mantém só as 50 revisões mais novas do projeto
DELETE FROM revisao
 WHERE projeto_id = $1
   AND numero <= (SELECT max(numero) FROM revisao
                  WHERE projeto_id = $1) - 50;
```

O texto é gravado no navegador enquanto você escreve. Para não encher o histórico com revisões quase iguais, a revisão automática é criada no máximo uma vez a cada 5 minutos de digitação; `Ctrl`+`S` sempre cria uma. No navegador, a poda das 50 mais recentes é feita logo depois de gravar a revisão nova.

Restaurar uma revisão

1. Abra **Arquivo › Revisões** (ou `GET /api/v1/projects/{id}/revisions`).
2. Escolha a revisão pela data e pelo total de palavras.
3. Clique em **Restaurar**. O projeto atual não é perdido: a restauração é um salvamento novo (com o próximo `numero`) e a versão anterior continua no histórico.

Capítulo 7

Concorrência otimista

O mesmo projeto pode estar aberto em duas abas ou em dois computadores ligados ao mesmo servidor. Para que um não apague o trabalho do outro sem aviso, cada gravação diz qual `versao` ela leu. O banco só aceita se ninguém salvou no meio do caminho. A coluna `versao` já vem no `schema.sql`.

```
UPDATE projeto
   SET titulo = $2, ..., versao = versao + 1, atualizado_em = now()
 WHERE id = $1
   AND versao = $3          -- versão que o editor leu
RETURNING versao;

-- na gravação com upsert, a mesma condição vai no DO UPDATE:
INSERT INTO projeto (id, titulo, autor, idioma, documento, versao)
VALUES ($1, $2, $3, $4, $5::jsonb, 1)
ON CONFLICT (id) DO UPDATE
   SET titulo = EXCLUDED.titulo, documento = EXCLUDED.documento,
       versao = projeto.versao + 1, atualizado_em = now()
 WHERE projeto.versao = $6
RETURNING versao;
```

Situação	Resultado
Ninguém salvou desde a leitura	1 linha atualizada; a API responde `200` com a `versao` nova
Outra aba ou pessoa salvou antes	0 linhas; `ROLLBACK` e a API responde `409 Conflict` com a versão atual
Projeto foi apagado	0 linhas e o projeto não existe: `404`

O que o editor faz num conflito

1. Mostra o aviso "Este projeto foi alterado em outro lugar".
2. Não sobrescreve nada: o texto continua salvo no navegador e a pessoa escolhe recarregar a versão mais nova ou salvar a sua como cópia (**Salvar como**). A versão de cada lado continua nas revisões.

No navegador o mesmo campo `versao` fica no store `projetos`: a transação do IndexedDB lê o registro, compara a versão e grava no mesmo passo, o que protege contra duas abas abertas no mesmo projeto.

Capítulo 8

Consultas prontas

Para rodar no `psql` ou em qualquer cliente PostgreSQL. Troque `:'projeto'` pelo id do projeto (no `psql`: `\set projeto '...'`).

Palavras por dia

```
SELECT s.inicio::date AS dia,
       SUM(GREATEST(s.palavras_fim - s.palavras_inicio, 0)) AS palavras,
       MAX(p.meta_diaria) AS meta,
       SUM(GREATEST(s.palavras_fim - s.palavras_inicio, 0)) >= MAX(p.meta_diaria) AS meta_batida
  FROM sessao_escrita s
  JOIN projeto p ON p.id = s.projeto_id
 WHERE s.projeto_id = :'projeto'
   AND s.inicio >= current_date - 30
 GROUP BY 1
 ORDER BY 1;
```

Capítulos por status

```
SELECT status, COUNT(*) AS capitulos, SUM(palavras) AS palavras
  FROM capitulo
 WHERE projeto_id = :'projeto'
 GROUP BY status
 ORDER BY array_position(ARRAY['rascunho','revisao','pronto'], status);
```

Últimas revisões

```
SELECT numero, criado_em,
       documento ->> 'title'                     AS titulo,
       jsonb_array_length(documento -> 'chapters') AS capitulos,
       tamanho
  FROM revisao
 WHERE projeto_id = :'projeto'
 ORDER BY numero DESC
 LIMIT 10;
```

Exportações por formato

```
SELECT formato, COUNT(*) AS vezes,
       pg_size_pretty(SUM(tamanho)::bigint) AS volume,
       MAX(criado_em) AS ultima
  FROM exportacao
 WHERE criado_em >= now() - interval '90 days'
 GROUP BY formato
 ORDER BY vezes DESC;
```

Projetos recentes com total de palavras

```
SELECT p.titulo, p.modo_formato, p.atualizado_em,
       COUNT(c.id) AS capitulos, COALESCE(SUM(c.palavras), 0) AS palavras
  FROM projeto p
  LEFT JOIN capitulo c ON c.projeto_id = p.id
 GROUP BY p.id
 ORDER BY p.atualizado_em DESC
 LIMIT 20;
```

Espaço ocupado

```
SELECT relname AS tabela, pg_size_pretty(pg_total_relation_size(relid)) AS tamanho
  FROM pg_catalog.pg_statio_user_tables
 ORDER BY pg_total_relation_size(relid) DESC;
```

Capítulo 9

Backup e restauração

Modo servidor: pg_dump

1. Gere a cópia com o servidor ligado (o `pg_dump` lê um retrato consistente):

```
pg_dump -h localhost -U serifa -d serifa -F c -f serifa-2026-10-28.dump
```

2. Guarde o arquivo `.dump` fora do computador (pen drive, nuvem pessoal). Ele contém os textos.
3. Para restaurar num banco vazio:

```
createdb -h localhost -U postgres -O serifa serifa
pg_restore -h localhost -U serifa -d serifa --no-owner serifa-2026-10-28.dump
```

4. Ligue o servidor e confira `http://127.0.0.1:8080/api/v1/health`.

> **Rotina sugerida.** Um `pg_dump` por dia (Agendador de Tarefas do Windows ou `cron`), guardando os últimos 14. No modo arquivos JSON, basta copiar a pasta `SERIFA_DATA`.

Qualquer modo: exportar .serifa

O arquivo `.serifa` é o backup mais simples e funciona em qualquer maneira de abrir o Serifa:

1. No editor, **Arquivo › Salvar como** (`Ctrl`+`Shift`+`S`) e escolha a pasta. No Chrome e no Edge, os próximos `Ctrl`+`S` gravam no mesmo arquivo.
2. Para restaurar, **Arquivo › Abrir** (`Ctrl`+`O`) e escolha o `.serifa`. Se já existir um projeto com o mesmo id, o Serifa pergunta se substitui ou abre como cópia.
3. Pela API: `GET /api/v1/projects/{id}` devolve o `.serifa`; `POST /api/v1/projects` com o arquivo no corpo importa (e já grava a primeira revisão).

O `.serifa` leva capítulos, notas, configurações, dicionário do projeto e ideias ignoradas. Não leva as revisões antigas nem as sessões de escrita.

Capítulo 10

Integridade e segurança

O que o banco guarda (e o que não guarda)

Guarda	Não guarda
Texto dos capítulos, títulos e notas	Nome real, e-mail, telefone ou endereço
Preferências do editor	Senhas ou contas (o Serifa não tem login)
Contagens de palavras e datas das sessões	Endereço IP, navegador ou localização
Autor como **pseudônimo opcional**	Qualquer dado pessoal obrigatório

Nenhum campo pessoal é obrigatório: `projeto.autor` aceita nulo e serve só para a capa da exportação (EPUB, Word). Também não há telemetria: nem o editor nem o servidor enviam dados para fora.

Proteções no servidor

- O servidor escuta só em `127.0.0.1` por padrão; para abrir na rede é preciso configurar de propósito.
- Todas as consultas usam `PreparedStatement` com parâmetros (nada de SQL montado com texto do usuário).
- Corpo das requisições limitado a 10 MB; HTML sanitizado antes de gravar (capítulo 3).
- Cabeçalhos de segurança: Content-Security-Policy, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` e `Referrer-Policy`.
- A senha do banco vem de variável de ambiente (`SERIFA_DB_PASSWORD`) e nunca aparece em log.

Usuário do banco com o mínimo de permissões

```
CREATE ROLE serifa LOGIN PASSWORD 'troque-esta-senha';
CREATE DATABASE serifa OWNER serifa;
-- depois de aplicar o schema.sql, conectado ao banco serifa:
REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO serifa;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO serifa;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO serifa;
```

Capítulo 11

Preparar o PostgreSQL

Só é necessário para o modo servidor com banco. O site e o `visualizar.bat` não usam PostgreSQL.

1. **Instale o PostgreSQL 15 ou mais novo** (no Windows, pelo instalador oficial; no Linux, pelo gerenciador de pacotes). Anote a senha do usuário `postgres`.
2. **Crie o usuário e o banco** do Serifa:

```
psql -h localhost -U postgres -c "CREATE ROLE serifa LOGIN PASSWORD 'troque-esta-senha';"
psql -h localhost -U postgres -c "CREATE DATABASE serifa OWNER serifa;"
```

3. **Aplique o esquema** (na raiz do projeto):

```
psql -h localhost -U serifa -d serifa -f server/src/main/resources/db/schema.sql
```

4. **Baixe o driver JDBC do PostgreSQL** (`postgresql-42.x.jar`) e deixe-o numa pasta conhecida, por exemplo `server/lib/`. O servidor não depende dele para compilar.
5. **Defina as variáveis de ambiente** (tabela abaixo) no terminal em que o servidor vai rodar.
6. **Compile e inicie o servidor** com o driver no classpath (`bash server/build.sh` ou `server\build.cmd` antes):

```
java -cp "server/target/serifa.jar:server/lib/postgresql-42.x.jar" serifa.cli.Main serve
rem no Windows o separador é ponto e vírgula:
java -cp "server\target\serifa.jar;server\lib\postgresql-42.x.jar" serifa.cli.Main serve
```

7. **Confira** `http://127.0.0.1:8080/api/v1/health`: deve responder `{"status":"ok","app":"Serifa",...}`.

Variáveis de ambiente

Variável	Exemplo	Para que serve
`SERIFA_DB_URL`	`jdbc:postgresql://localhost:5432/serifa`	Liga o modo PostgreSQL. Sem ela, o servidor usa arquivos JSON
`SERIFA_DB_USER`	`serifa`	Usuário do banco
`SERIFA_DB_PASSWORD`	`troque-esta-senha`	Senha do usuário (nunca grave no repositório)
`SERIFA_DATA`	`./dados-servidor`	Pasta dos arquivos JSON, quando não há banco
`SERIFA_PORT`	`8080`	Porta do servidor

Windows (PowerShell)

```
$env:SERIFA_DB_URL = "jdbc:postgresql://localhost:5432/serifa"
$env:SERIFA_DB_USER = "serifa"
$env:SERIFA_DB_PASSWORD = "troque-esta-senha"
```

Linux e macOS

```
export SERIFA_DB_URL="jdbc:postgresql://localhost:5432/serifa"
export SERIFA_DB_USER="serifa"
export SERIFA_DB_PASSWORD="troque-esta-senha"
```

Solução de problemas

Sintoma	O que fazer
O servidor diz que está usando arquivos JSON	`SERIFA_DB_URL` não foi definida no mesmo terminal, ou o driver JDBC não está no classpath
`password authentication failed`	Confira `SERIFA_DB_USER` e `SERIFA_DB_PASSWORD`; teste com `psql -U serifa -d serifa`
`relation "projeto" does not exist`	O `schema.sql` não foi aplicado nesse banco (passo 3)
`409 Conflict` ao salvar	Outra aba salvou antes; veja o capítulo 7
`Connection refused`	O PostgreSQL não está rodando ou não escuta na porta 5432